

Universidad “Mayor de San Andrés”

FACULTAD DE CIENCIAS PURAS Y NATURALES



Trabajo Individual

Integrante:

Cantuta Quispe Elvis Alberth

Carrera: Informática

Docente: Ph.D. MOISES MARTIN SILVA CHOQUE

Fecha: 15 de junio de 2025

La Paz-Bolivia

a) Clasificación de Texturas

Desarrollar una aplicación capaz de diferenciar superficies dentro de una imagen, por ejemplo:

césped, tierra, cemento o asfalto, aplicando principios similares a los utilizados en clasificación

de imágenes satelitales.

Objetivos Específicos

- Cargar imágenes digitales desde el sistema.
- Analizar las características cromáticas de cada píxel.
- Evaluar la textura local mediante una vecindad de 5x5 píxeles.
- Clasificar las superficies utilizando reglas heurísticas.
- Generar una imagen temática donde cada categoría sea representada por un color específico.

Herramientas Utilizadas

Lenguaje de Programación

- C#

Framework

- .NET Windows Forms

Librerías

- System
- System.Drawing
- System.Windows.Forms

La metodología utilizada consiste en analizar cada píxel de la imagen original y determinar a qué categoría pertenece mediante reglas basadas en color y textura.

El proceso se divide en las siguientes etapas:

Etapas 1: Carga de Imagen

El usuario selecciona una imagen desde su computadora mediante un cuadro de diálogo.

La imagen seleccionada se almacena en un objeto Bitmap para permitir el acceso a cada píxel.

Etapas 2: Recorrido Pixel a Pixel

El sistema recorre toda la imagen analizando individualmente cada píxel.

Para evitar errores en los bordes se deja un margen de dos píxeles alrededor de la imagen.

Etapas 3: Análisis de Textura

Para cada píxel se analiza una vecindad de 5x5 píxeles.

Se calcula:

- Brillo promedio.
- Variación del brillo.

La variación permite determinar si una superficie es lisa o rugosa.

Ejemplos:

- Agua → Baja variación.
- Cemento → Variación baja.
- Vegetación → Variación media.
- Tierra → Variación alta.

Etapas 4: Conversión al Modelo HSV

La clasificación utiliza el modelo de color HSV:

Hue (Tono)

Representa el color dominante.

Ejemplos:

- Verde → Vegetación.
- Azul → Agua.
- Marrón → Tierra.

Saturation (Saturación)

Representa la intensidad del color.

Valores altos indican colores vivos.

Valores bajos indican colores grises.

Brightness (Brillo)

Representa la cantidad de luz presente en el píxel.

Algoritmo de Clasificación

La clasificación se realiza mediante reglas heurísticas basadas en los valores HSV.

Agua

Características:

- Tonos azules.
- Saturación media o alta.

Clasificación:

Color Azul.

Vegetación

Características:

- Tonos verdes.
- Saturación media o alta.

Clasificación:

Color Verde.

Tierra

Características:

- Tonos marrones o anaranjados.
- Saturación elevada.

Clasificación:

Color Marrón.

Líneas de Carretera

Características:

- Saturación muy baja.
- Brillo elevado.

Clasificación:

Color Blanco.

Asfalto

Características:

- Saturación baja.
- Brillo medio o bajo.

Clasificación:

Color Gris Oscuro.

Cemento

Características:

- Saturación baja.
- Brillo alto.

Clasificación:

Color Gris Claro.

Sombras

Características:

- Brillo muy bajo.

Clasificación:

Color Negro.

Desconocido

Características:

- No cumple ninguna regla de clasificación.

Clasificación:

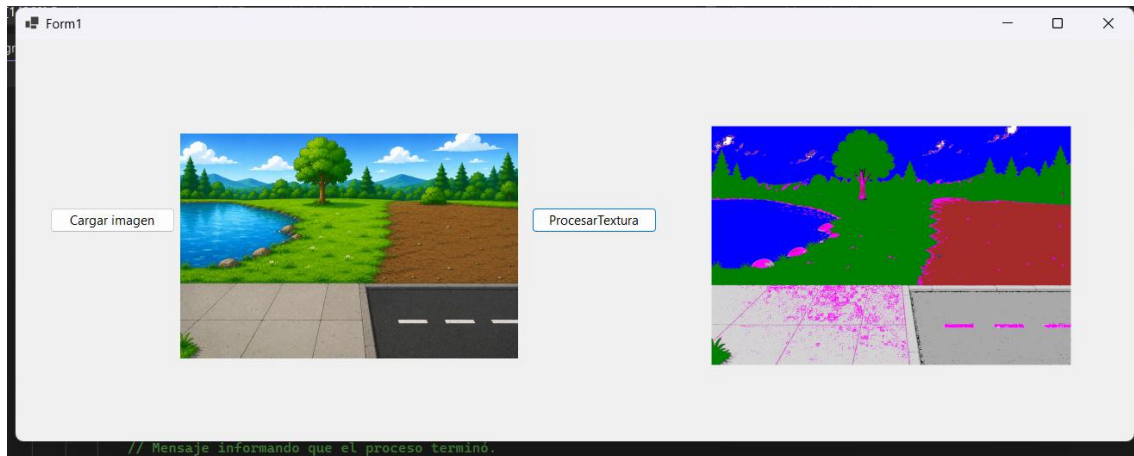
Color Magenta.

Resultados Esperados

La imagen procesada genera un mapa temático donde cada tipo de superficie es representado mediante un color específico.

Superficie	Color
Agua	Azul
Vegetación	Verde
Tierra	Marrón
Asfalto	Gris Oscuro
Cemento	Gris Claro
Líneas Viales	Blanco
Sombras	Negro

Superficie	Color
Desconocido	Magenta



```
using System;
using System.Drawing;
using System.Windows.Forms;

namespace Prueba
{
    public partial class Form1 : Form
    {
        // Constructor del formulario
        // Se ejecuta al iniciar la aplicación y carga los componentes gráficos.
        public Form1()
        {
            InitializeComponent();

            // =====
            // BOTÓN CARGAR IMAGEN
            // =====
            // Permite al usuario seleccionar una imagen desde su computadora
            // para posteriormente analizarla y clasificar sus superficies.
            private void btnCargar_Click(object sender, EventArgs e)
            {
                OpenFileDialog buscarImagen = new OpenFileDialog();

                // Filtra únicamente archivos de imagen
                buscarImagen.Filter = "Archivos de Imagen|*.jpg;*.jpeg;*.png;*.bmp";

                // Si el usuario selecciona una imagen
                if (buscarImagen.ShowDialog() == DialogResult.OK)
                {
                    // Se carga la imagen en el PictureBox original
                    picOriginal.Image = new Bitmap(buscarImagen.FileName);
                }
            }

            // =====
            // BOTÓN PROCESAR
            // =====
            // Analiza cada píxel de la imagen para identificar
            // diferentes tipos de superficies:
            // Agua, vegetación, tierra, asfalto y cemento.
            private void btnProcesar_Click(object sender, EventArgs e)
            {
                // Mensaje informando que el proceso terminó.
            }
        }
    }
}
```

```

{
    // Verifica que exista una imagen cargada
    if (picOriginal.Image == null)
    {
        MessageBox.Show(";Primero debes cargar una imagen!");
        return;
    }

    // Convierte la imagen cargada en un objeto Bitmap
    // para poder acceder a cada uno de sus píxeles.
    Bitmap imgOriginal = new Bitmap(picOriginal.Image);

    // Crea una imagen vacía donde se almacenará
    // el resultado de la clasificación.
    Bitmap imgResultado = new Bitmap(
        imgOriginal.Width,
        imgOriginal.Height);

    // =====
    // RECORRIDO PIXEL A PIXEL
    // =====
    // Se recorren todos los píxeles de la imagen.
    // Se dejan 2 píxeles de margen porque se utiliza
    // una ventana de análisis de textura de 5x5.
    for (int x = 2; x < imgOriginal.Width - 2; x++)
    {
        for (int y = 2; y < imgOriginal.Height - 2; y++)
        {
            // Obtiene el color del píxel actual
            Color pixelActual = imgOriginal.GetPixel(x, y);

            // =====
            // ANÁLISIS DE TEXTURA
            // =====
            // La textura permite conocer qué tan uniforme
            // o rugosa es una zona de la imagen.
            //
            // Una superficie lisa (agua o cemento)
            // tiene poca variación.
            //
            // Una superficie rugosa (tierra o vegetación)
            // tiene mayor variación.
            double sumaBrillos = 0;

            // Recorremos una vecindad de 5x5 alrededor
            // del píxel actual.
            for (int kx = -2; kx <= 2; kx++)
            {
                for (int ky = -2; ky <= 2; ky++)
                {
                    Color vecino =
                        imgOriginal.GetPixel(x + kx, y + ky);

                    // Se acumula el brillo de cada vecino.
                    sumaBrillos += vecino.GetBrightness();
                }
            }

            // Calcula el brillo promedio de la ventana.
            double promedioBrillo = sumaBrillos / 25.0;

            // Calcula la variación del brillo.
            double variacion = 0;

            for (int kx = -2; kx <= 2; kx++)
            {

```

```

        for (int ky = -2; ky <= 2; ky++)
        {
            Color vecino =
                imgOriginal.GetPixel(x + kx, y + ky);

            variacion += Math.Abs(
                vecino.GetBrightness()
                - promedioBrillo);
        }
    }

    // =====
    // EXTRACCIÓN DE CARACTERÍSTICAS HSV
    // =====
    // HSV permite identificar colores de forma
    // más precisa que RGB.
    //
    // Hue (Tono): Color principal.
    // Saturation: Intensidad del color.
    // Brightness: Brillo.
    float hue = pixelActual.GetHue();
    float sat = pixelActual.GetSaturation();
    float bri = pixelActual.GetBrightness();

    Color colorClasificado;

    // =====
    // CLASIFICACIÓN DE SUPERFICIES
    // =====

    // AGUA
    // Generalmente presenta tonos azules.
    if (hue >= 180 && hue <= 250 &&
        sat > 0.35)
    {
        colorClasificado = Color.Blue;
    }

    // VEGETACIÓN
    // Tonos verdes con saturación media o alta.
    else if (hue >= 70 &&
        hue <= 170 &&
        sat > 0.25)
    {
        colorClasificado = Color.Green;
    }

    // TIERRA
    // Tonos marrones o anaranjados.
    else if (hue >= 15 &&
        hue <= 40 &&
        sat > 0.30)
    {
        colorClasificado = Color.Brown;
    }

    // LÍNEAS BLANCAS DE LA CARRETERA
    // Son colores poco saturados y muy brillantes.
    else if (sat < 0.10 &&
        bri > 0.80)
    {
        colorClasificado = Color.White;
    }

    // ASFALTO
    // Color gris oscuro y bajo brillo.

```



```

        else if (sat < 0.15 &&
                bri > 0.15 &&
                bri < 0.45)
        {
            colorClasificado = Color.DarkGray;
        }

        // CEMENTO
        // Similar al asfalto pero más claro.
        else if (sat < 0.15 &&
                bri >= 0.45)
        {
            colorClasificado = Color.LightGray;
        }

        // SOMBRAS
        // Regiones muy oscuras.
        else if (bri < 0.15)
        {
            colorClasificado = Color.Black;
        }

        // DESCONOCIDO
        // Cualquier píxel que no cumpla
        // las reglas anteriores.
        else
        {
            colorClasificado = Color.Magenta;
        }

        // Guarda el resultado de la clasificación
        // en la nueva imagen.
        imgResultado.SetPixel(
            x,
            y,
            colorClasificado);
    }
}

// Muestra la imagen clasificada
// en el PictureBox de resultados.
picResultado.Image = imgResultado;

// Mensaje informando que el proceso terminó.
MessageBox.Show("¡Clasificación completada!");
}
}
}

```

Ejercicio 2

b) Implementación de un Filtro de Suavizado Diseñar un filtro de promedio que recorra la imagen utilizando ventanas de 3×3 píxeles para reducir ruido y suavizar transiciones de color.

El procesamiento digital de imágenes permite mejorar la calidad visual de una imagen mediante la aplicación de diferentes técnicas y filtros. Uno de los filtros más utilizados es el filtro de suavizado por promedio, cuyo objetivo es reducir el ruido presente en una imagen y generar transiciones más suaves entre los píxeles.

En este proyecto se desarrolló una aplicación utilizando C# y Windows Forms capaz de cargar una imagen, aplicar un filtro de suavizado mediante una ventana de 3x3 píxeles y mostrar el resultado obtenido.

Implementar un filtro de suavizado por promedio que reduzca el ruido y las variaciones bruscas de color en una imagen digital.

Objetivos Específicos

- Cargar imágenes digitales desde el sistema.
- Analizar los píxeles vecinos de cada posición.
- Calcular el promedio de los componentes RGB.
- Generar una nueva imagen suavizada.
- Visualizar el resultado obtenido.

3. Herramientas Utilizadas

Lenguaje de Programación

- C#

Framework

- Windows Forms (.NET)

Librerías

- System
- System.Drawing
- System.Windows.Forms

Etapas 2: Creación de la Imagen Resultado

Se crea una nueva imagen vacía con las mismas dimensiones que la imagen original.

Esta imagen almacenará los resultados del filtro aplicado.

Etapas 3: Recorrido Pixel a Pixel

El algoritmo recorre toda la imagen utilizando dos ciclos:

- Filas (eje Y)
- Columnas (eje X)

Se deja un margen de un píxel alrededor de la imagen para evitar errores al analizar la vecindad de 3x3.

Etapla 4: Ventana Deslizante 3x3

Para cada píxel se analiza una región de nueve píxeles:

P1 P2 P3

P4 P5 P6

P7 P8 P9

donde P5 representa el píxel central.

Etapla 5: Cálculo del Promedio

Se calculan por separado los promedios de los componentes:

- Rojo (R)
- Verde (G)
- Azul (B)

Fórmula utilizada

Para el canal rojo:

$$\text{PromedioR} = \Sigma R / 9$$

Para el canal verde:

$$\text{PromedioG} = \Sigma G / 9$$

Para el canal azul:

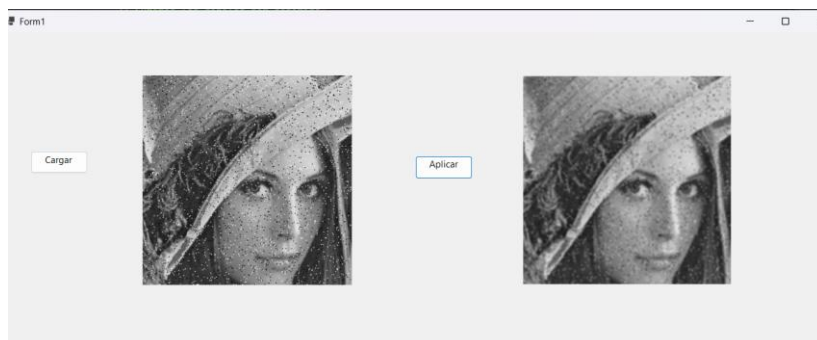
$$\text{PromedioB} = \Sigma B / 9$$

Los tres valores obtenidos forman el nuevo color del píxel.

Etapla 6: Generación de la Imagen Suavizada

El nuevo color promedio se almacena en la imagen resultado.

Este proceso se repite para todos los píxeles de la imagen.



```

using System;
using System.Drawing;
using System.Windows.Forms;

namespace FiltroSuavizadoPromedio
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            // Código para el botón de Cargar Imagen
            private void btnCargar_Click(object sender, EventArgs e)
            {
                OpenFileDialog buscarImagen = new OpenFileDialog();
                buscarImagen.Filter = "Archivos de Imagen|*.jpg;*.jpeg;*.png;*.bmp";

                if (buscarImagen.ShowDialog() == DialogResult.OK)
                {
                    picOriginal.Image = new Bitmap(buscarImagen.FileName);
                }
            }

            // Código para el algoritmo del Filtro de Promedio (Suavizado)
            private void btnSuavizar_Click(object sender, EventArgs e)
            {
                if (picOriginal.Image == null)
                {
                    MessageBox.Show("Primero debes cargar una imagen!");
                    return;
                }

                // 1. Convertimos la imagen cargada en un objeto Bitmap para leer
                sus píxeles
                Bitmap imgOriginal = new Bitmap(picOriginal.Image);

                // 2. Creamos un mapa de bits vacío del mismo tamaño para el
                resultado suavizado
                Bitmap imgResultado = new Bitmap(imgOriginal.Width,
                imgOriginal.Height);

                // 3. Recorremos la imagen píxel por píxel con bucles for (Filas y
                Columnas)
                // Dejamos 1 píxel de margen en los bordes para que la ventana 3x3
                no se salga de la matriz
                for (int x = 1; x < imgOriginal.Width - 1; x++)
                {
                    for (int y = 1; y < imgOriginal.Height - 1; y++)
                    {
                        // Variables para acumular (sumar) los colores de los 9
                        píxeles vecinos
                        int sumaR = 0;
                        int sumaG = 0;
                        int sumaB = 0;

                        // 4. Ventana deslizante de 3x3 alrededor del píxel central
                        (x, y)
                        for (int kx = -1; kx <= 1; kx++)
                        {
                            for (int ky = -1; ky <= 1; ky++)
                            {
                                // Obtenemos el color exacto del píxel vecino a
                                nivel de byte

```

```

        Color pixelVecino = imgOriginal.GetPixel(x + kx, y +
ky);

        // Sumamos los canales por separado
        sumaR += pixelVecino.R;
        sumaG += pixelVecino.G;
        sumaB += pixelVecino.B;
    }
}

// 5. Calculamos el promedio matemático dividiendo el total
entre los 9 píxeles analizados
int promedioR = sumaR / 9;
int promedioG = sumaG / 9;
int promedioB = sumaB / 9;

// 6. Creamos el nuevo color promediado (suavizado)
Color colorSuavizado = Color.FromArgb(promedioR, promedioG,
promedioB);

// 7. Asignamos este color al píxel correspondiente en la
nueva imagen
imgResultado.SetPixel(x, y, colorSuavizado);
}
}

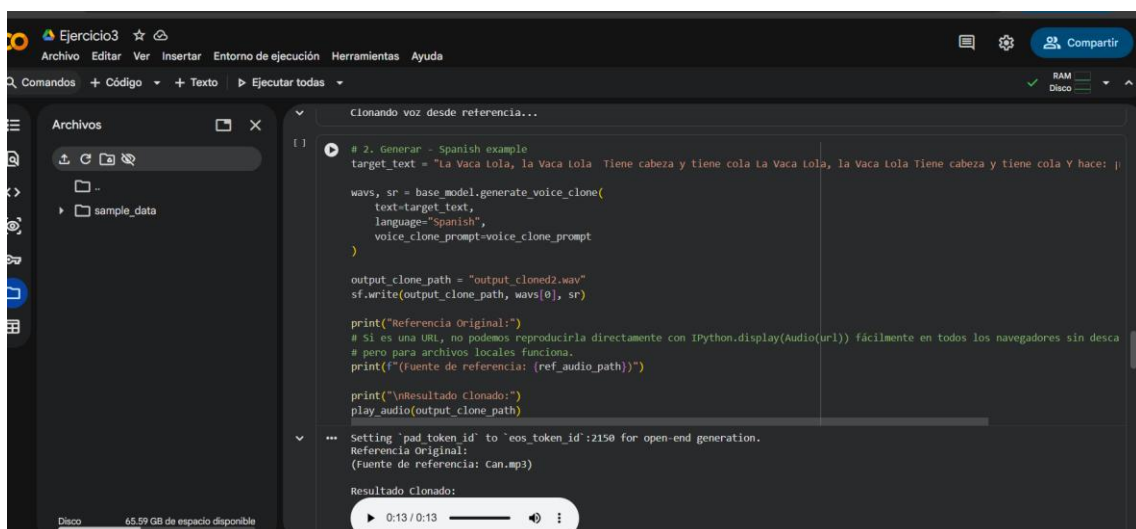
// 8. Mostramos el resultado final en la pantalla
picResultado.Image = imgResultado;
MessageBox.Show("¡Filtro de suavizado (promedio) aplicado con
éxito!");
}
}
}

```

c) Cover de "La Vaca Lola"

Realizar una producción multimedia de la canción infantil utilizando la identidad de un artista al menos conocido, Vincular el audio con una animación (puede ser el modelo 3D o los monigotes) asegurando que el ritmo y el movimiento coincidan.

Primero Realizamos el audio de un cantante famoso, en este caso use el del CANCERBERO



Luego Tenemos que hacer la música para el fondo

```
catall todas
[ ] ▶ carpeta = "/content/drive/MyDrive/multimedia"
      os.makedirs(carpeta, exist_ok=True)

      synthesiser = pipeline(
          "text-to-audio",
          "facebook/musicgen-small",
          device=0
      )

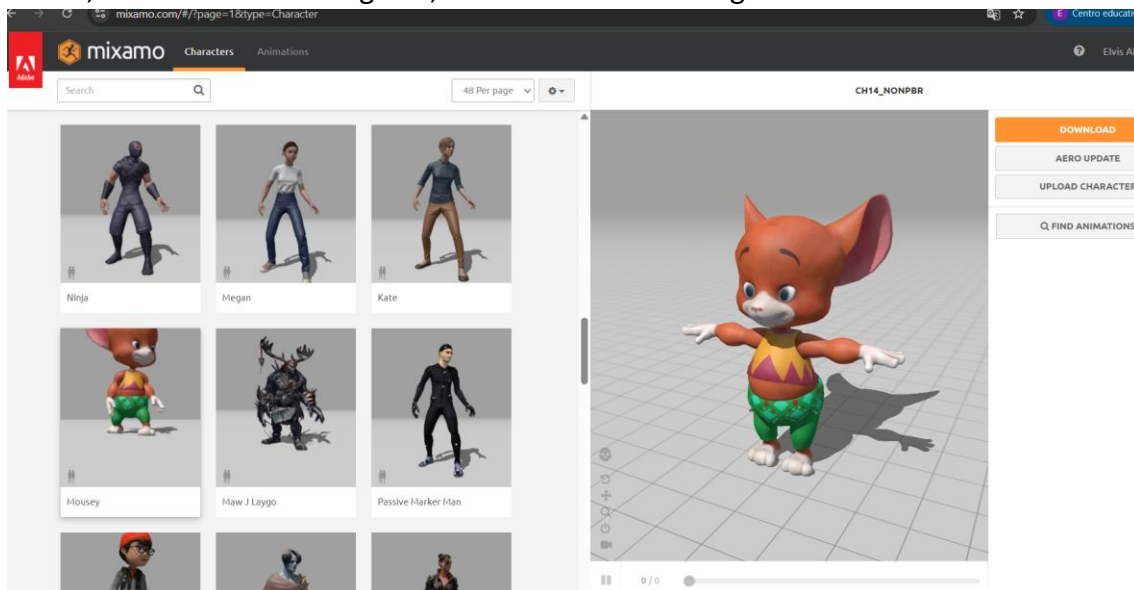
      prompt = """
      Cancion infantil alegre, estilo pop latino divertido,
      ritmoailable para niños, instrumentos suaves,
      melodía simple, ambiente feliz, inspirada en una canción alegre.
      """

      music = synthesiser(
          prompt,
          forward_params={
              "do_sample": True,
              "max_new_tokens": 1500
          }
      )

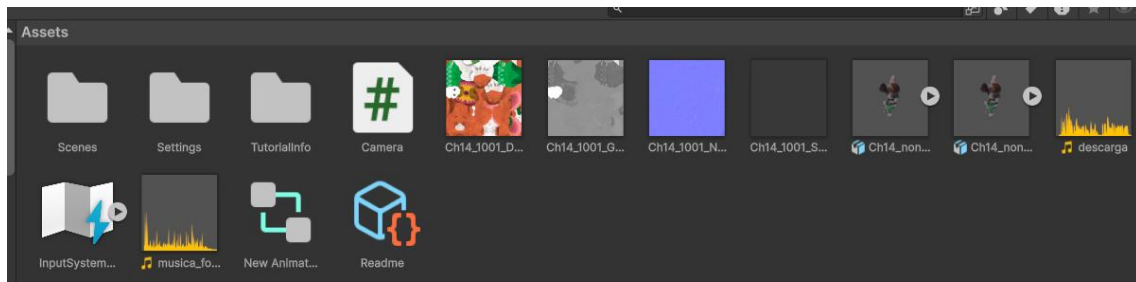
      ruta_salida = carpeta + "/musica_fondo_30s.wav"

      scipy.io.wavfile.write(
          ruta_salida,
          rate=music["sampling_rate"],
          data=music["audio"]
      )
```

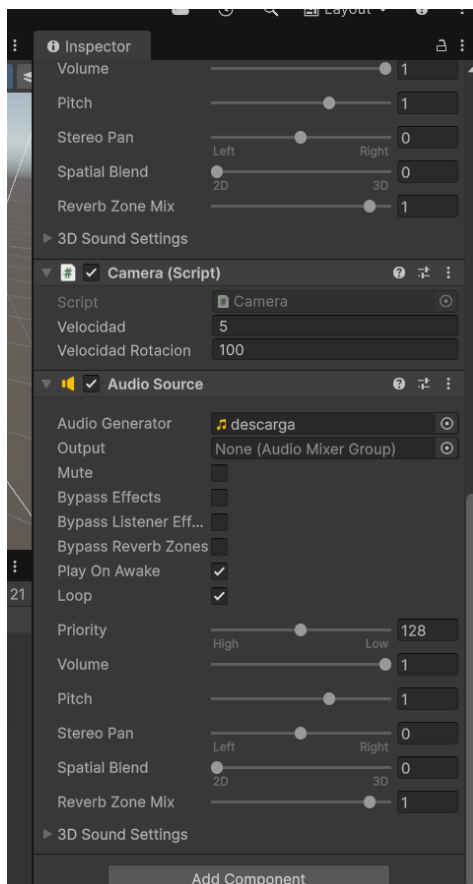
Ahora, vamos con los monigotes, en este caso lo descargamos desde Mixamo



Y lo pasamos a Unity al apartado de assets



Lo movemos junto con la música y la textura



Creamos un script para poder mover la cámara a la hora de correr el programa

```
using UnityEngine;

public class Camera : MonoBehaviour
{
    public float velocidad = 5.0f;
    public float velocidadRotacion = 100.0f;

    void Start()
    {

    }

    void Update()
    {
        // Movimiento con flechas o WASD
        float movimientoHorizontal = Input.GetAxis("Horizontal");
        float movimientoVertical = Input.GetAxis("Vertical");

        // Subir y bajar con E y Q
        float movimientoArribaAbajo = 0;

        if (Input.GetKey(KeyCode.E))
        {
            movimientoArribaAbajo = 1;
        }

        if (Input.GetKey(KeyCode.Q))
        {
            movimientoArribaAbajo = -1;
        }

        // Direccion de movimiento de la camara
        Vector3 direccion = new Vector3(
            movimientoHorizontal,
            movimientoArribaAbajo,
            movimientoVertical
        );

        // Mover la camara
        transform.Translate(direccion * velocidad * Time.deltaTime);

        // Rotar la camara con el mouse manteniendo clic derecho
        if (Input.GetMouseButton(1))
        {
            float rotacionHorizontal = Input.GetAxis("Mouse X");
            float rotacionVertical = Input.GetAxis("Mouse Y");
        }
    }
}
```



```
        transform.Rotate(Vector3.up, rotacionHorizontal *  
velocidadRotacion * Time.deltaTime, Space.World);  
        transform.Rotate(Vector3.left, rotacionVertical *  
velocidadRotacion * Time.deltaTime, Space.Self);  
    }  
}  
}
```